DO COMANDO AO SCRIPT

*Seis comandos toda manhã, sempre na mesma ordem. Enquanto a sequência só existe nos dedos de quem digita, ela é um **hábito** — não um procedimento. O script é o terminal gravado.*

051
GUIAS LINUX

DO ENTER AO PROCESSO — O QUE ACONTECE EM `./ROTINA.SH`

1 · VOCÊ
`./rotina.sh`
digita e tecla Enter.

→

2 · SHELL
`fork() + execve()`
cria o filho e pede ao kernel para executar o arquivo.

→

3 · KERNEL
`#!`
lê os **dois primeiros bytes**. Não é binário: é *shebang*.

→

4 · KERNEL
`/bin/bash`
carrega o interpretador da linha e lhe entrega o arquivo.

→

5 · BASH
`linha a linha`
lê *enquanto* executa — não valida o arquivo antes.

O passo 3 é a razão de o bit `x` ser obrigatório: sem ele o `execve()` falha **antes** de qualquer leitura. Sem *shebang* algum, o kernel devolve `Exec format error` e o Bash interpreta por cortesia histórica — cortesia que o `cron` e o `systemd` não têm. E, como o passo 5 lê por posição de byte, **erro de sintaxe na linha 50 não impede as linhas 1 a 49 de rodarem**: valide com `bash -n` antes da primeira execução.

EXECUTAR × `SOURCE` — A DIFERENÇA QUE EXPLICA O `.BASHRC`

./SCRIPT.SH

- ✓ **Processo novo.** O filho recebe uma *cópia* do ambiente; ao terminar, a cópia é descartada.
- ✓ Exige bit `x` e *shebang*. `bash script.sh` dispensa os dois.
- ✓ `cd`, `export` e funções **não sobrevivem**: o shell que chamou fica como estava.
- ✓ Depurar: `bash -x script.sh` mostra cada linha expandida.

SOURCE SCRIPT.SH

- ✓ **Mesmo processo.** As linhas são lidas como se tivessem sido digitadas — `$$` não muda.
- ✓ Forma POSIX: `. script.sh`. Não exige bit `x`.
- ✓ Altera a sessão em curso: PWD, variáveis e funções passam a valer.
- ✓ É o mecanismo do `.bashrc` e do `/etc/profile.d/`. Por isso `bash ~/.bashrc` "não faz nada".

O *SHEBANG* CERTO E OS TRÊS ERROS CLÁSSICOS

PRIMEIRA LINHA	O QUE SIGNIFICA
`#!/usr/bin/env bash`	Procura o `bash` no `PATH`. **Padrão portável** — roda em Debian, macOS, BSD.
`#!/bin/bash`	Caminho fixo e absoluto. Use quando você controla o sistema e quer garantia de qual binário roda.
`#!/bin/sh`	No Debian/Kali é o **dash**: sem `[[ ]]`, arrays ou `local`. Em RHEL é o Bash em modo POSIX — o mesmo arquivo funciona lá e falha aqui.

`Permission denied`
Achou o arquivo, falta o bit `x`. → `chmod +x`

`command not found`
O arquivo está fora do `PATH`. → `./` na frente, ou instale em `~/bin`.

`/usr/bin/env^M: bad interpreter`
Editado no Windows: fim de linha CRLF. → `dos2unix` ou `sed -i 's/\r$//'`

```
kali@kali: ~/lab-scripts
$ cat > rotina.sh <<'FIM'      # aspas em FIM: nada é expandido agora
#!/usr/bin/env bash            # 1. quem interpreta
# rotina.sh - conferencia diaria | Uso: ./rotina.sh   # 2. o que faz e como usar
set -euo pipefail              # 3. -e para no 1o erro · -u variavel indefinida · pipefail
DESTINO="$HOME/relatorios"     # 4. config no topo, SEMPRE entre aspas
mkdir -p "$DESTINO"
df -h /  >  "$DESTINO/sis.txt" # 5. > cria/zera · >> acrescenta
free -h  >> "$DESTINO/sis.txt"
FIM
$ bash -n rotina.sh              # valida a sintaxe sem executar NADA
$ chmod +x rotina.sh && ./rotina.sh
```

Nunca acrescente `.` ao `PATH`. O `PATH="$PATH:."` no `.bashrc` troca segurança por conveniência: basta plantar um arquivo chamado `ls`, `git` ou `sudo` num diretório compartilhado para que o próximo comando digitado ali execute código alheio. A solução correta é `mkdir -p ~/bin` — o `.profile` do Debian/Kali já o acrescenta ao `PATH` se ele existir. E **nunca edite um script longo enquanto ele roda**: o Bash lê por posição de byte e continuará a partir do deslocamento antigo, agora no meio de outra linha. Edite uma cópia e troque com `mv`.

RESUMO RÁPIDO

- ✓ **Script é o terminal gravado.** Mesma sintaxe, mesmo interpretador — monte o pipeline no terminal, confira a saída, e só então copie para o arquivo (`history 20 | cut -c 8-` é o rascunho).
- ✓ **O *shebang* diz quem interpreta**, e o bit `x` autoriza o kernel a sequer olhar o arquivo. São dois requisitos, não um.
- ✓ **Executar cria processo; `source` não.** Só `source` muda a sessão atual.
- ✓ **Esqueleto decente em 5 itens:** *shebang* · cabeçalho de comentário · `set -euo pipefail` · variáveis de config no topo · toda expansão entre aspas.
- ✓ `bash -n` antes da primeira execução. Custa nada e o shell não valida sozinho.

NO KALI

O `~/bin` já está previsto — mas só se você criá-lo. O `.profile` só o acrescenta ao `PATH` *se o diretório existir*, e a checagem roda no login: você cria a pasta, põe o script lá e nada acontece até `source ~/.profile`. Para ferramenta própria de uso geral, a convenção é `sudo install -m 755 meu.sh /usr/local/bin/meu` — diretório que nenhum pacote do APT toca. **O shell padrão do Kali é o Zsh desde 2020**, mas isso não muda nada em script: com `#!/usr/bin/env bash` no topo ele roda em Bash. O erro é testar a linha no Zsh interativo, ver funcionar e supor que valerá no script — confira com `bash -c '...'`. Script que embrulha ferramenta ofensiva precisa ser **mais** explícito: valide a lista de alvos, exiba-a e exija confirmação antes de agir — agir fora do escopo autorizado é crime (Lei 12.737/2012) e "o script leu o arquivo errado" não é defesa. Para aprender convenção lendo gente experiente: `less $(which kali-tweaks)`.